Ray在蚂蚁AI Infra生态的实践与探索

About Me

祝森林

蚂蚁集团高级技术专家
Ray团队负责人

2018年初加入蚂蚁集团以来一直专注于通用分布式计算引擎Ray的研发，结合蚂蚁规模化业务场景重塑Ray在蚂蚁的底层架构，对Ray内核模块及其设计有深入理解，对Ray上支撑规模化应用有较丰富的实战经验。

1 Ray的起源 & 价值

2 Ray AI Infra生态能力建设

3 Ray在蚂蚁的实践与探索

01

Ray的起源 & 价值

■ 洞见

- ✓ 未来AI系统将持续与环境交互，并从中自我进化
- ✓ 新一代AI应用在性能和灵活性上，对系统提出新的需求

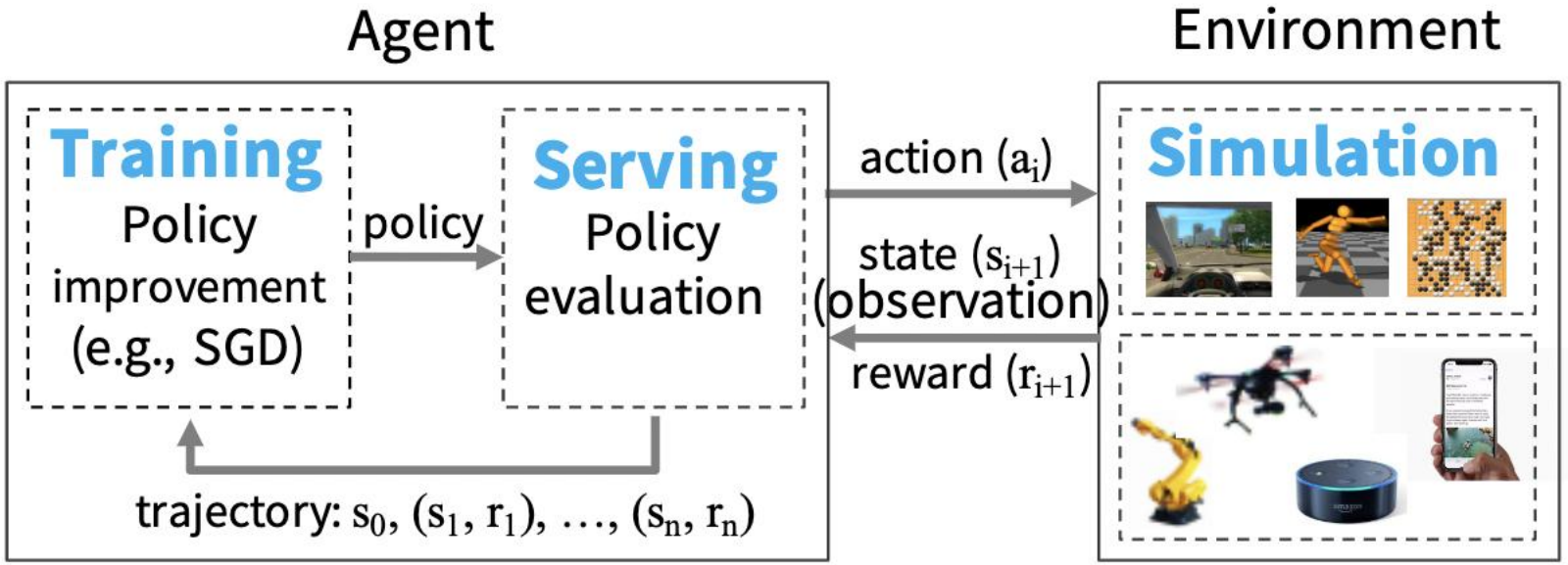
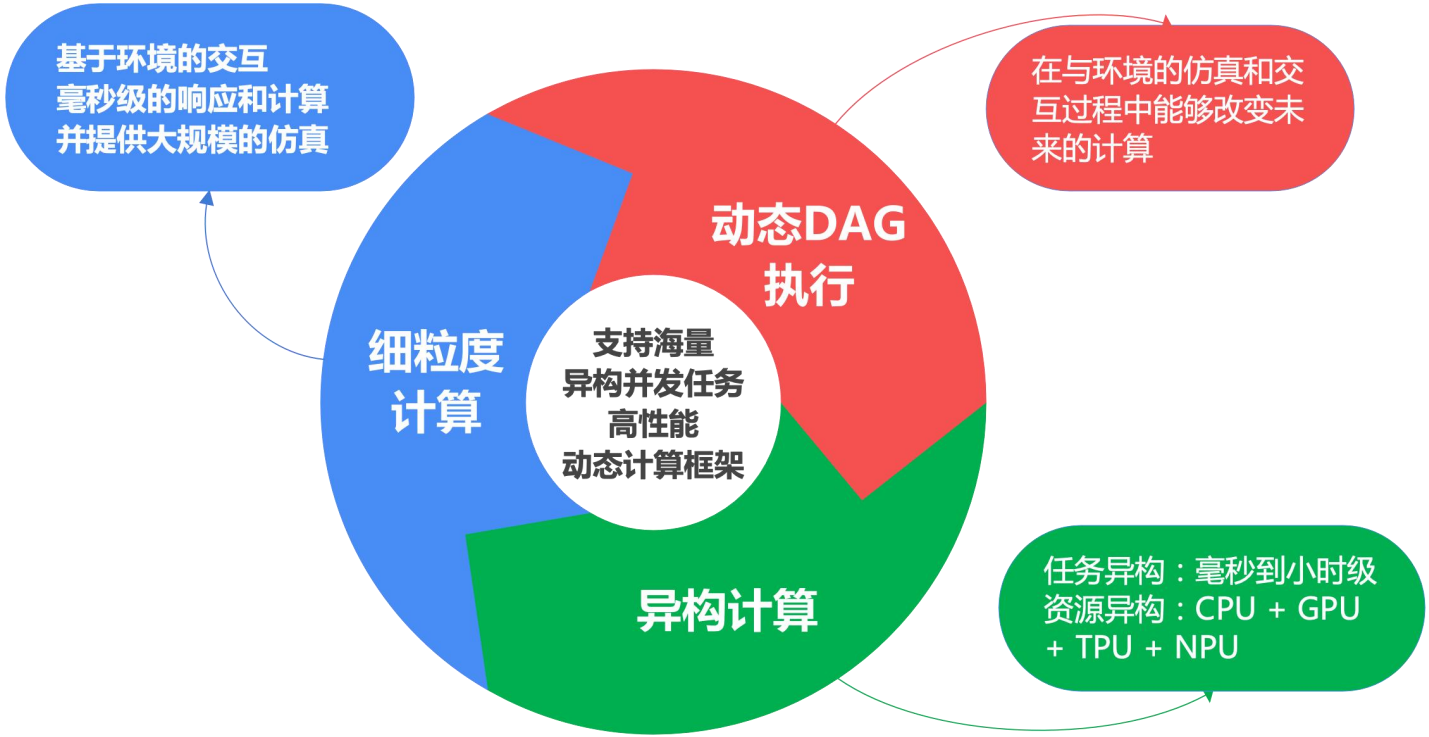


Figure 1: Example of an RL system.

■ 系统能力需求


基于环境的交互
毫秒级的响应和计算
并提供大规模的仿真
在与环境的仿真和交互过程中能够改变未来的计算
细粒度计算
动态DAG执行
异构计算
支持海量
异构并发任务
高性能
动态计算框架
任务异构：毫秒到小时级
资源异构：CPU + GPU + TPU + NPU

当时的技术现状和挑战

系统类型	典型框架	挑战【不支持】
BSP-Synchronous Parallel System	MapReduce ApacheSpark Dryad	细粒度的仿真 策略服务
Task Parallelism System	CIEL Dask	分布式训练 策略服务
Streaming System	Naiad Storm	分布式训练 策略服务
Distributed Deep-Learning Frameworks	TF MXNet	细粒度的仿真 策略服务
Model-Serving	TF-Serving Clipper	训练 仿真

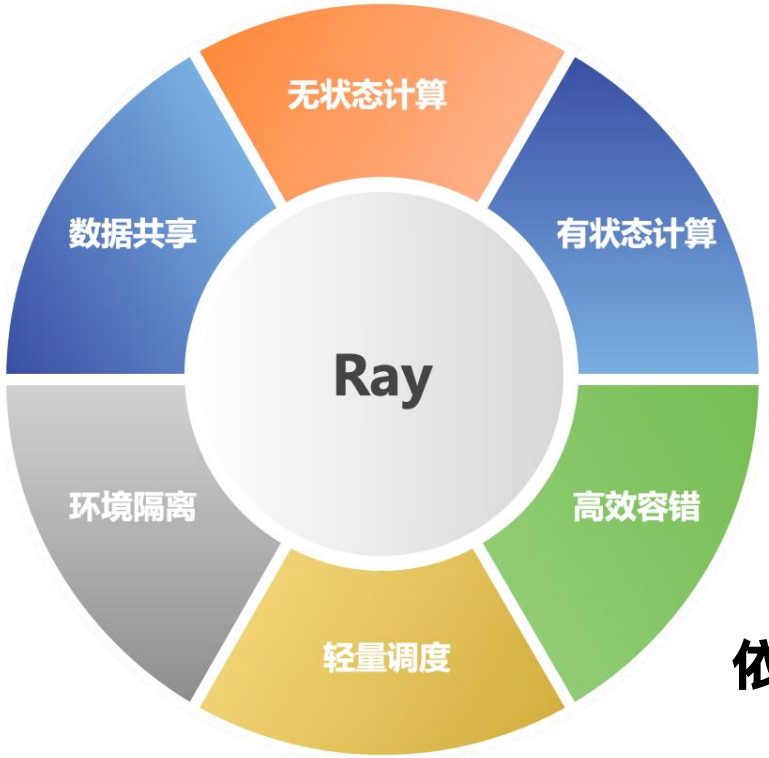
基于多套系统进行组合拼接，在多系统之间调度、容灾和数据移动

组合方案

融合方案

构建一个通用的计算框架去解决仿真、处理、训练及服务

■ 构建统一分布式编程框架



Key idea：不绑定计算模式，

把单机编程中的基本概念分布式化

无状态	Function	→	RayTask
有状态	Class	→	RayActor
依赖及输出	Object	→	RayObject

■ 编程模型

蚂蚁集团 ANT GROUP | 支付宝

单机（Function）

```python
def heavy_compute(value):
    # Heavy computation here.
    # return ...

res = [heavy_compute(i)
       for i in range(10000)]
print(res)
```

分布式（Task）

```python
@ray.remote
def heavy_compute(value):
    # Heavy computation here.
    # return ...

res = [heavy_compute.remote(i)
       for i in range(10000)]
print(ray.get(res))
```

把普通function变成remote task

异步调用，远程执行

获取结果

单机（Class）

```python
class Counter:

    def __init__(self):
        self.value = 0

    def increment(self):
        self.value += 1
        return self.value
```

分布式（Actor）

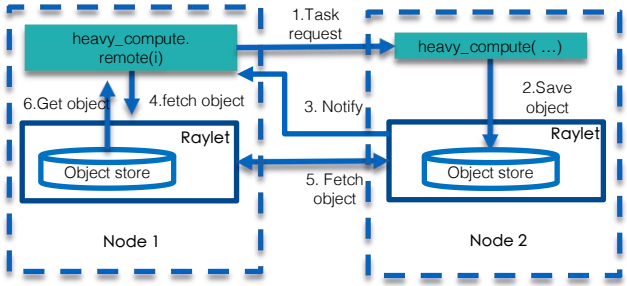
```python
@ray.remote
class Counter:
    # ...

counter = Counter.remote()

res = [counter.increment.remote()
       for _ in range(3)]

assert ray.get(res) == [1, 2, 3]
```

把普通Class变成Actor Class

创建远程Actor对象，返回Actor Handle

远程调用Actor方法

计算依赖及结果数据共享（Object）



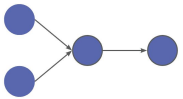

```python
@ray.remote
def heavy_compute(value):
    # Heavy computation here.
    # return ...

res = [heavy_compute.remote(i)
       for i in range(10000)]
print(ray.get(res))
```

ObjectRef

■ 计算模型

蚂蚁集团 ANT GROUP | 支付宝

静态图

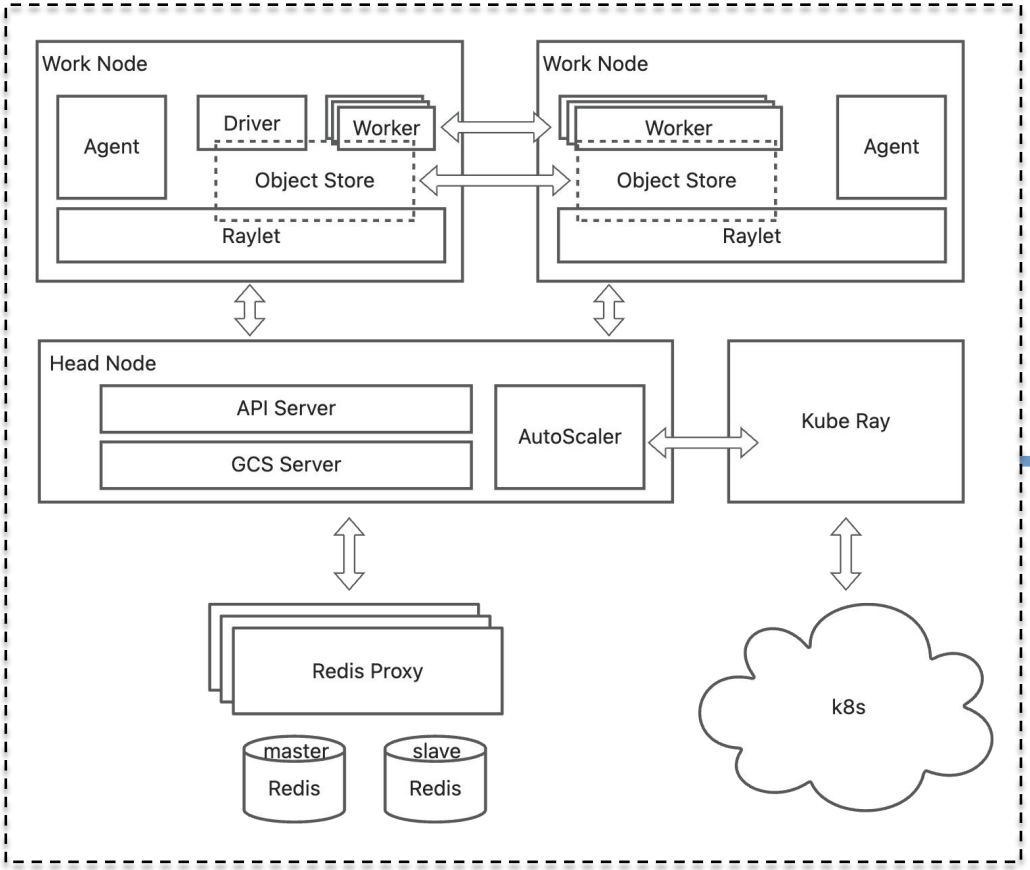


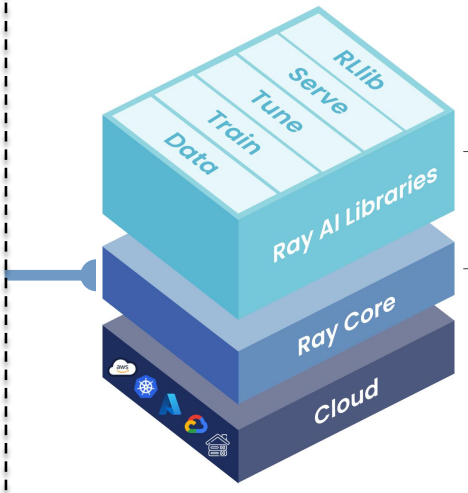
- ✓ **特点**：计算流程预先确定，任务依赖关系执行前已知
- ✓ **场景**：批处理、预定义数据处理流水线
- ✓ **优势**：可进行全局优化，任务调度更高效

动态图

- ✓ **特点**：计算流程运行时确定，任务可动态创建
- ✓ **场景**：强化学习、实时决策系统、交互式应用
- ✓ **优势**：灵活性高，可根据中间结果动态调整执行路径

RAY框架





high-level libraries that enable simple scaling of AI workloads

a low-level distributed computing framework with a concise core and Python-first API

■ RAY官方定义

蚂蚁集团 ANT GROUP | 支付宝

Evolution

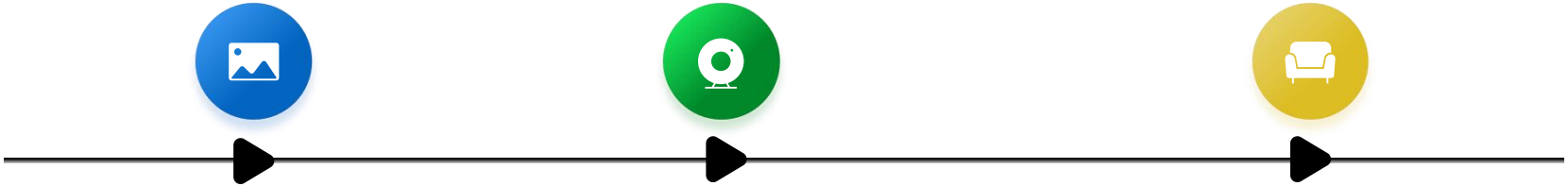
0.x Ray is a flexible, high-performance distributed execution framework.

1.x Ray provides a simple, universal API for building distributed applications.

2.x Ray is a unified framework for scaling AI and Python applications, Ray consists of a core distributed runtime and a set of AI libraries for simplifying ML compute.

我们的思考——灵魂三问

蚂蚁集团 ANT GROUP | 支付宝



Ray到底是什么

提供一整套面向AI全栈的分布式框架底座，让AI系统从单机到分布式。

Ray到底能做什么

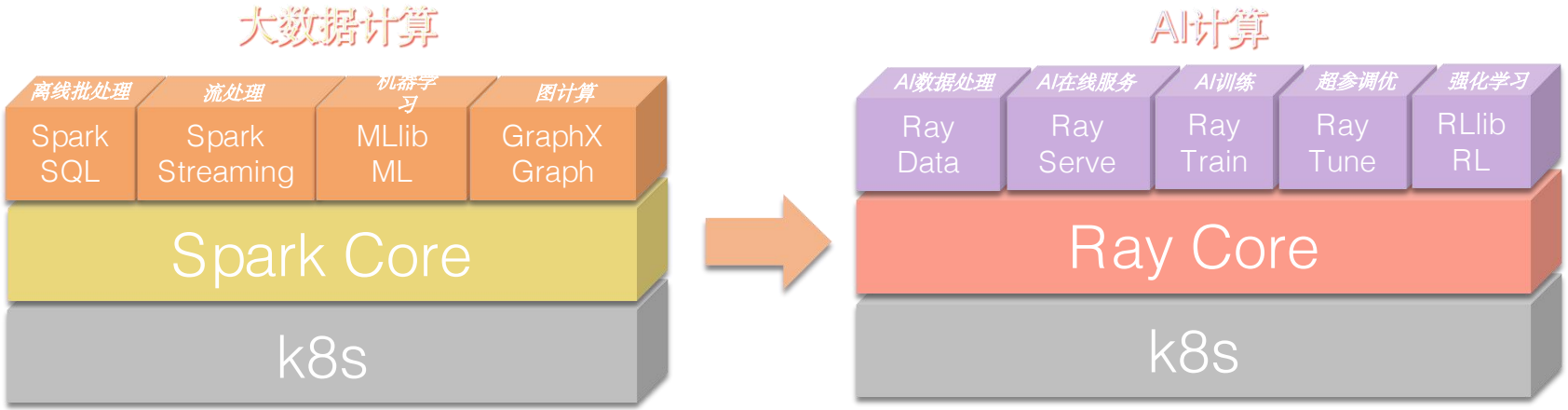
和整个AI的开源生态天然的融合，围绕Ray Core构建从训练、推理到在线服务一整套AI基建，让“用户”快速构建自己的AI Infra体系。

Ray这几年到底做对了什么

- Python生态缺乏一个工业级的分布式框架（同时运行时是基于C++这种Native语言）
- 优秀的编程模型抽象(Actor & Task)，从无状态的计算到有状态的计算。
- 从Ray Core到Ray AI Libraries，从单个引擎内核到AI生态体系化组件库的能力建设，让用户简单、快速的构建自己的AI Infra基座。

■ 我们的思考——计算范式的迭代和演进

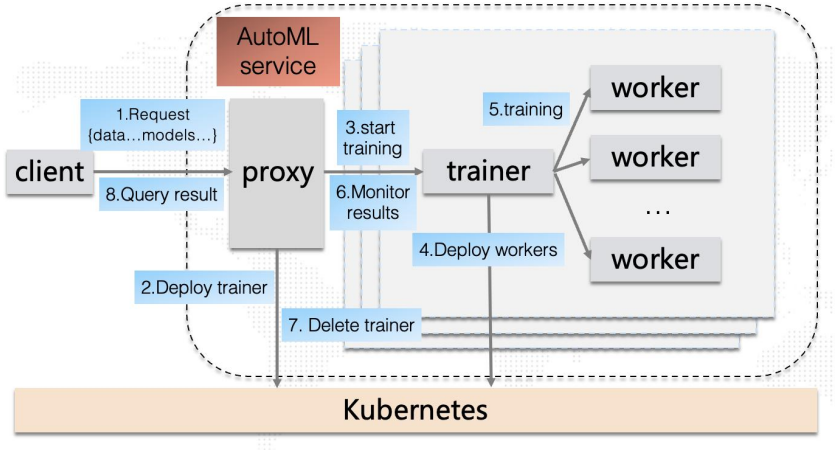
- 面向大数据场景，Spark定义了一套通用的数据计算框架
- 面向AI场景，Ray定义了一套统一的AI计算框架



■ 我们的思考——Ray和K8s到底区别在哪里？

蚂蚁集团 ANT GROUP | 支付宝

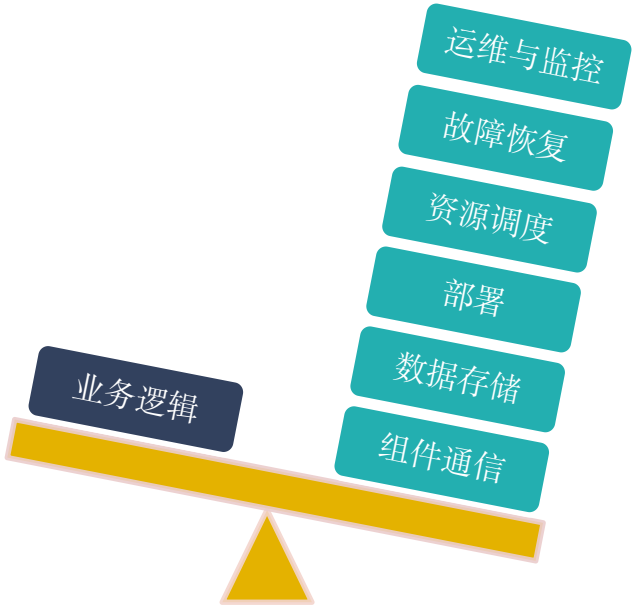
在K8S和训练引擎(TensorFlow、Pytorch)、推理引擎(TensorFlow-Serving、vLLM)之间需要一个Ray。
(K8S: Kubernetes, also known as K8s, is an open source system for automating deployment, scaling, and management of containerized applications)



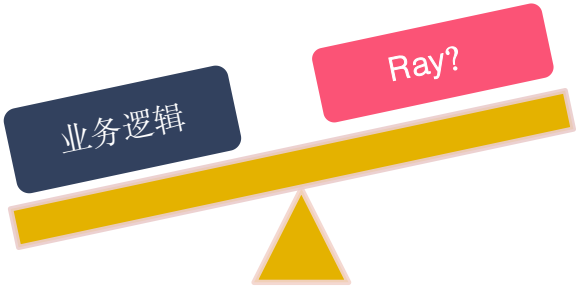
	云原生方式	Ray的方式
研发效率	15人天	2人天
代码	Python 800行 Protobuf 100行 Dockerfile 50行 Go 1100行 YAML 300行	Python 260 行
工程特性	多语言 多程序入口 应用、运维部署、配置分离	单语言 一个main函数，单程序入口 应用、运维部署、配置融为一体

完整Demo地址: https://github.com/alipay/antray/tree/compare/examples/automl
博文: https://ray.osanswer.net/t/topic/296

■ RAY的价值



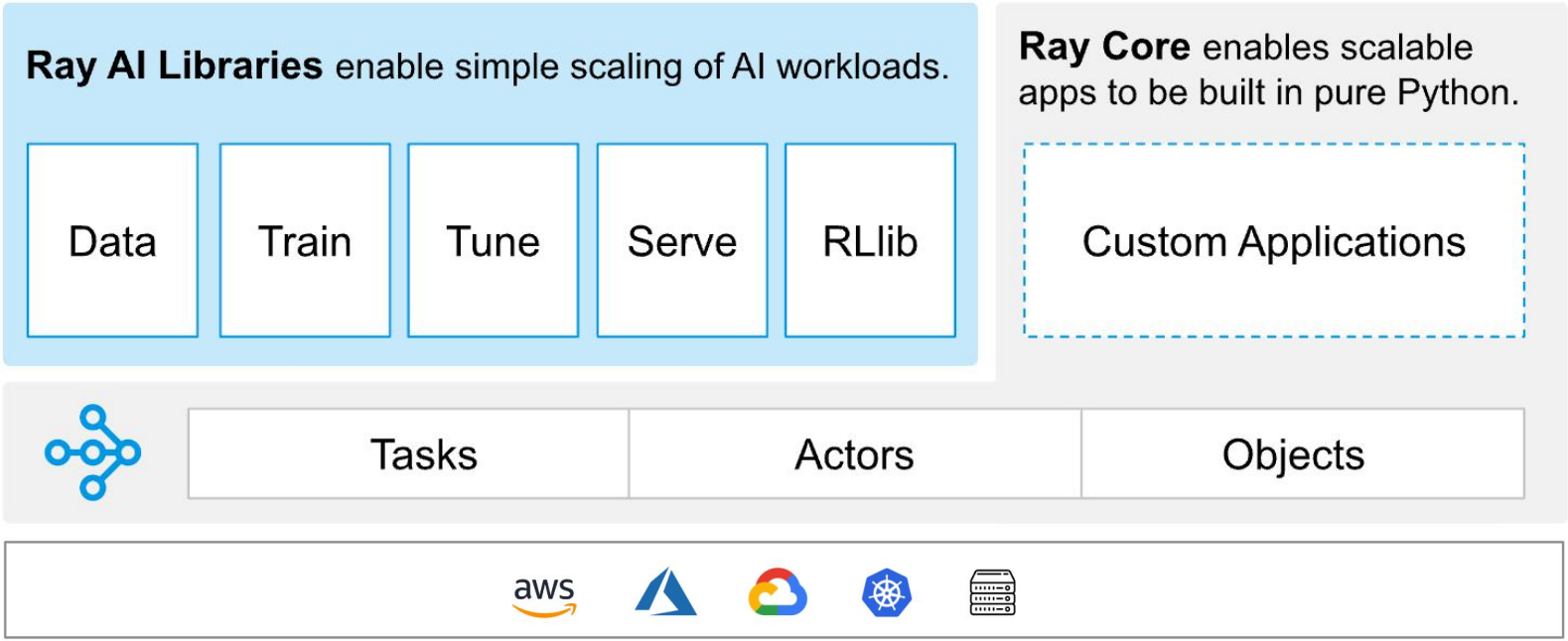
- 屏蔽分布式系统底层复杂细节
- 上层专注于平台本身，把平台做厚，把系统摊薄



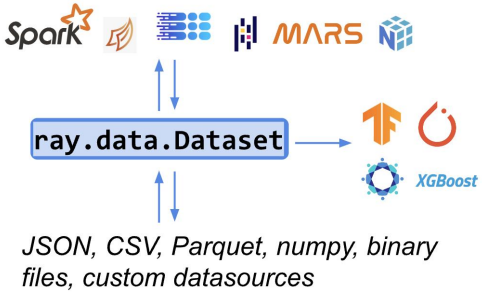
02

Ray AI Infra生态能力建设

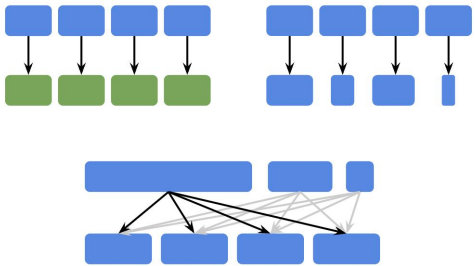
Ray AI Libraries



Ray AI Libraries —— Ray Data



Standard way to load and exchange data in Ray



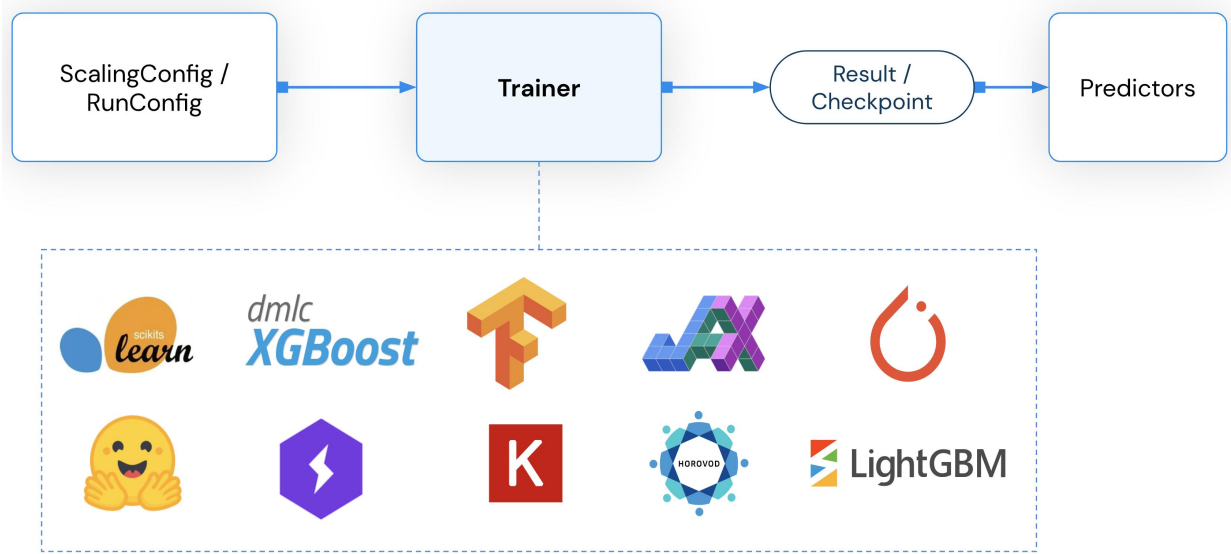
Basic distributed ops: map, filter, and repartition

```
ds = ray.data.read_parquet(...)
# Pass datasets to tasks and actors
func.remote(ds)
# Split datasets into shards
shards = ds.split(xgboost.num_workers,
    locality_hints=xgboost.workers)
# Distributed training on datasets
for i, s in enumerate(shards):
   xgboost.workers[i].train.remote(s)
```

Seamless interop with Ray tasks, actors, and libraries

- 应用于AI数据处理
- Ray AI数据加载以及AI生态库交换数据的关键桥梁
- 支持主流数据格式加载：json、csv、parquet、numpy、pandas、huggingface等
- 提供基本的分布式算子：map、filter、sort、repartition等
- 与Ray的tasks、actors以及libraries无缝兼容

■ Ray AI Libraries —— Ray Train



对多种机器学习框架扩展训练进行了抽象，提供3类Trainer:

- Deep Learning Trainers（Pytorch、Tensorflow、Horovod)
- Tree-based Trainers（Xgboost、LightGBM)
- Other ML frameworks （HuggingFace、Scikit-Learn、RLlib)

■ Ray AI Libraries —— Ray Tune

Ray tune是**超参调优**的Python库

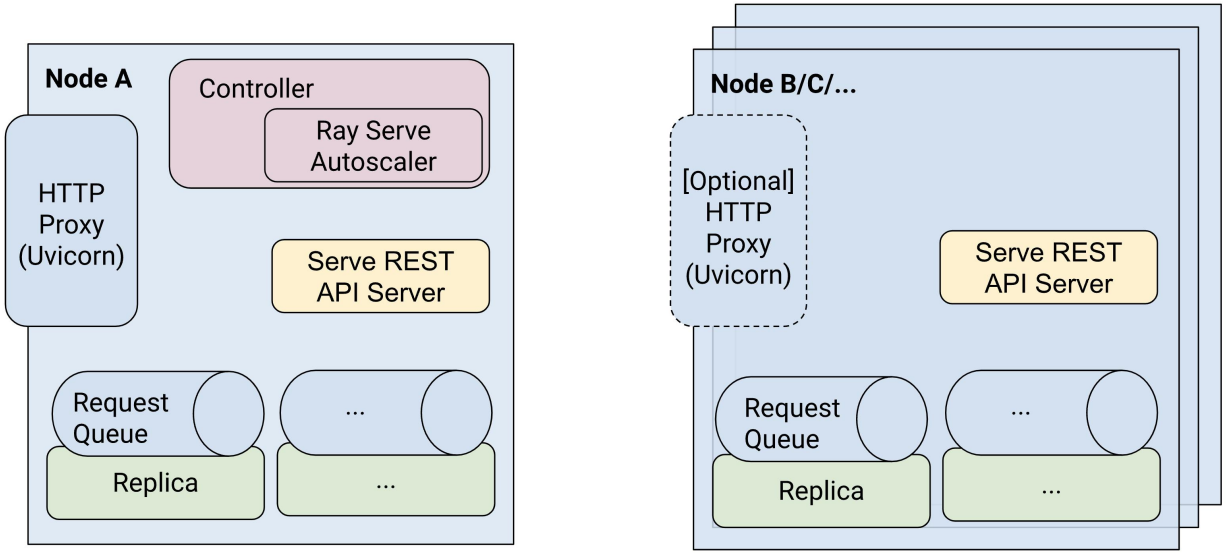
支持多种ML框架

- PyTorch
- XGBoost
- Scikit-Learn
- TensorFlow
- Keras

支持多种超参数调优工具

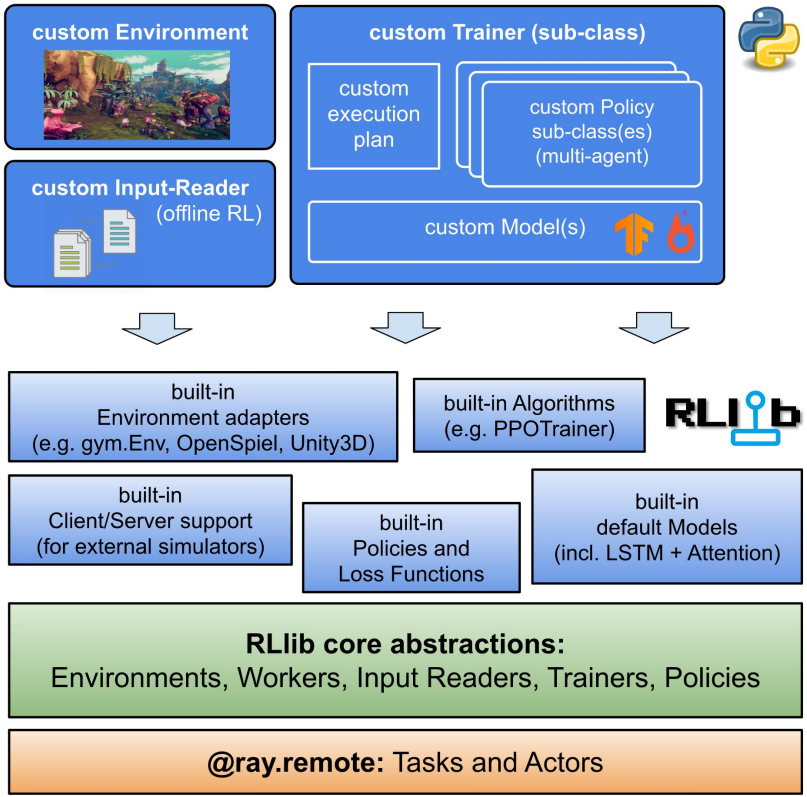
Ax	HEBO
BayesOpt	FLAML
BOHB	Optuna
Dragonfly	SigOpt
Hyperopt	Skopt
Nevergrad	ZOOpt

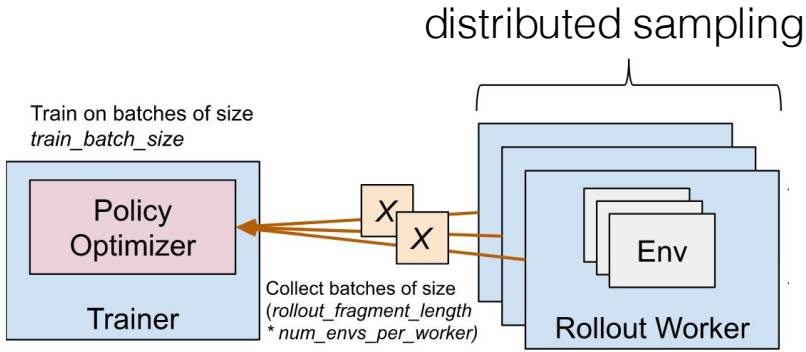
■ Ray AI Libraries —— Ray Serve



- Ray Serve 是一个可扩展的模型服务库，用于构建在线推理服务
- 能够构建由多个 ML 模型+业务逻辑组成的复杂推理服务
- 建立在 Ray 之上，可以轻松扩展到许多机器并提供灵活的异构资源调度支持
- 支持生成式AI的流式响应（Stream Response）

Ray AI Libraries —— RLlib



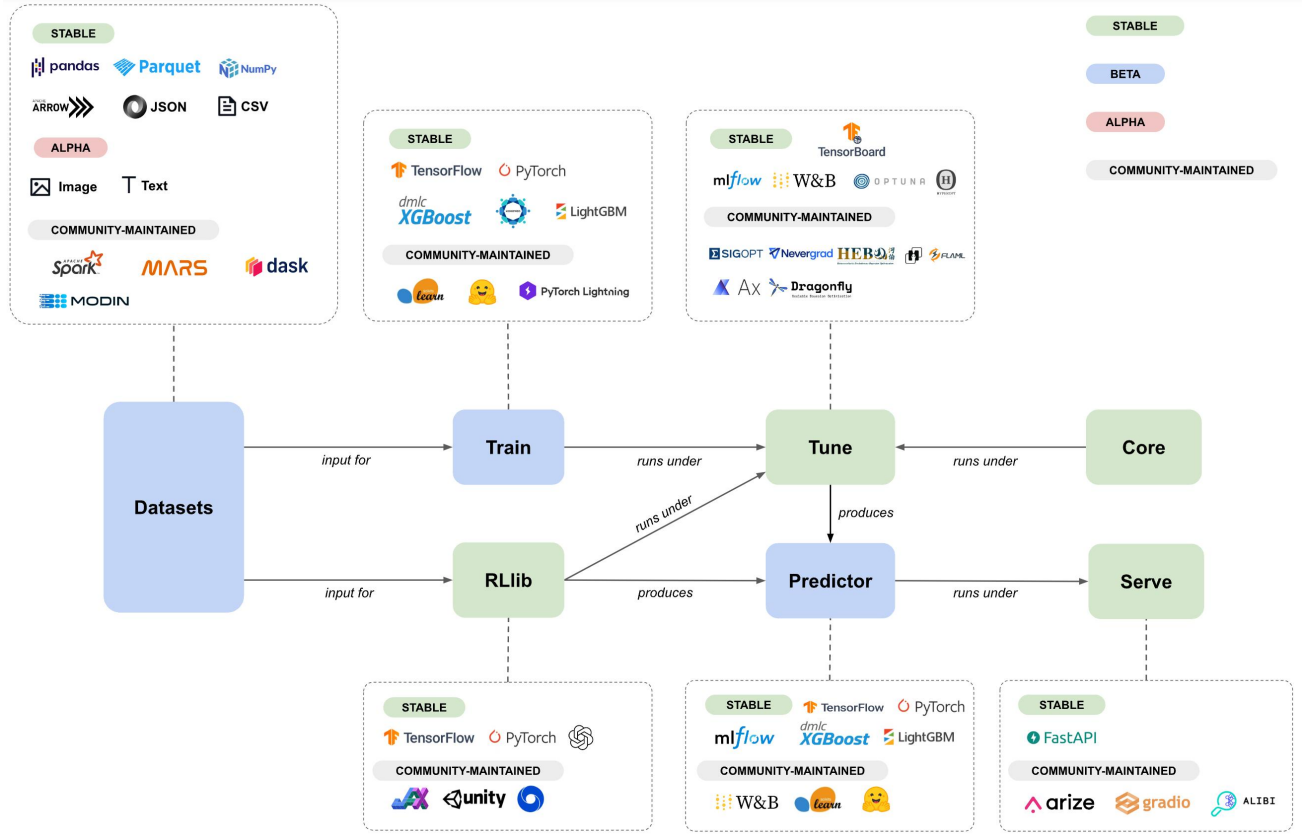


内置丰富的开箱即用RL算法

High-level抽象，易于定制和扩展

支持大规模分布式RL训练和分布式采样

More AI Libraries

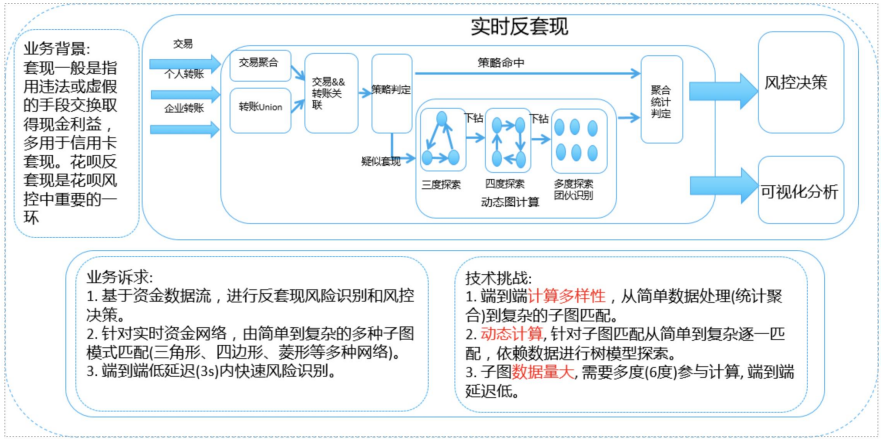


03

Ray在蚂蚁实践与探索

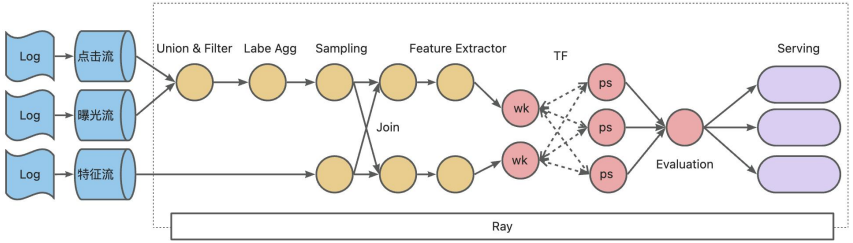
■ 早期的探索

动态DAG



图计算GeaFlow

融合计算



在线学习 ODL

■ 规模化增长

蚂蚁集团 ANT GROUP | 支付宝

蚂蚁Ray团队成立

专注通用分布式引擎底座

蚂蚁对Ray内核贡献超26%

同时落地菜鸟和达摩院

聚焦强化学习场景

围绕训推一体强化场景

推出可视化平台工具链

在线学习引擎落地

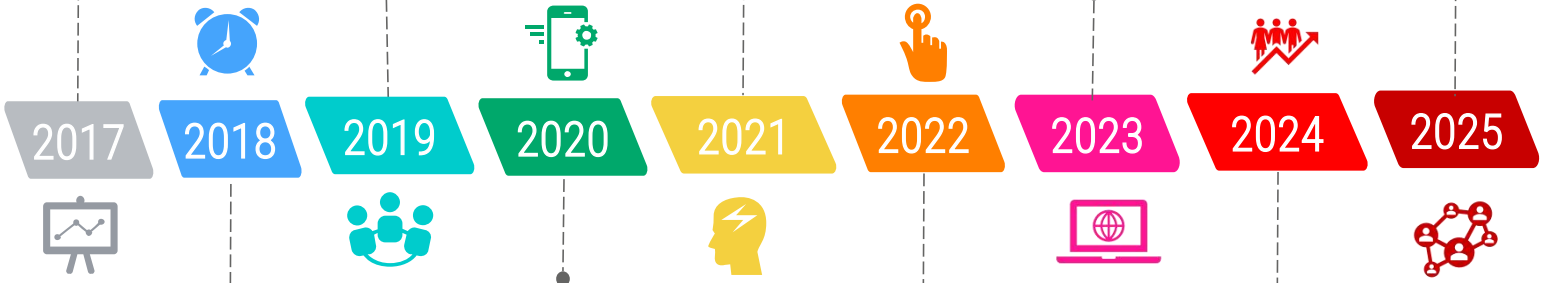
流计算、模型训练、

模型部署一体化

统一AI服务框架

统一推理、大模型和

搜索引擎的在线服务



流图计算Geaflow落地

安全风控场景

多租户架构

运筹优化、科学计

算、在线推理落地

隐私计算框架"隐语"落地
FaaS函数计算平台落地

在南网、浦发、中信等企业

商业化输出

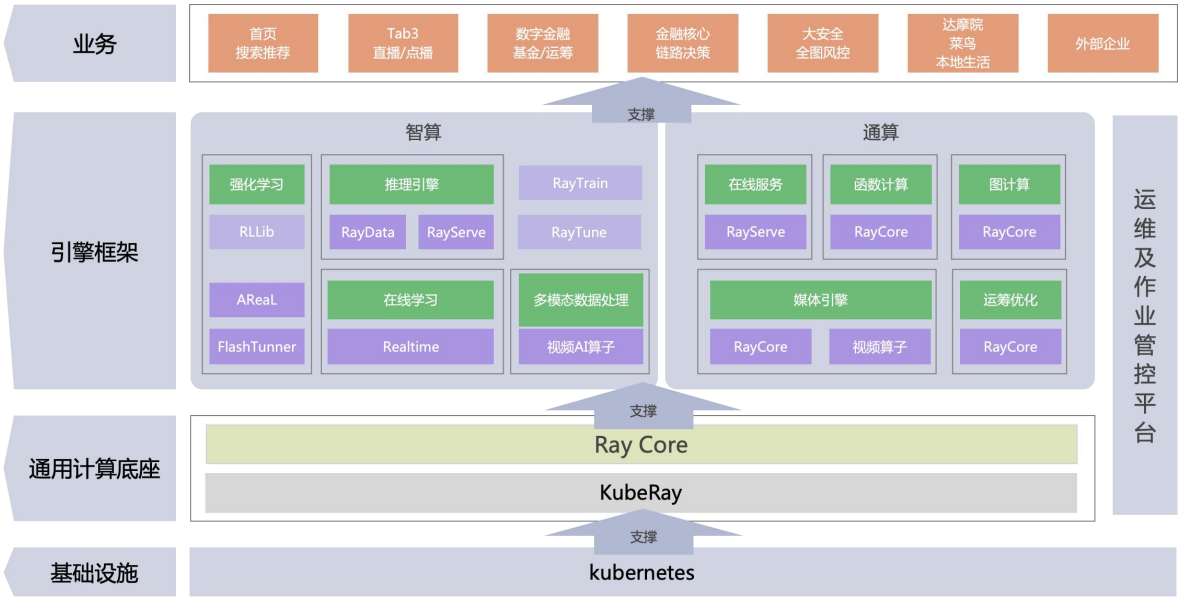
离在线一体AI计算平台

打造离在线一体推理架构

■ 蚂蚁RAY生态架构

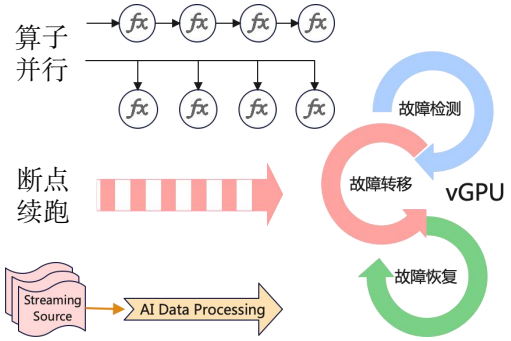
蚂蚁集团 ANT GROUP | 支付宝

- 基础库：社区原生 + 内部扩展
- 引　擎：智算类 + 通算类

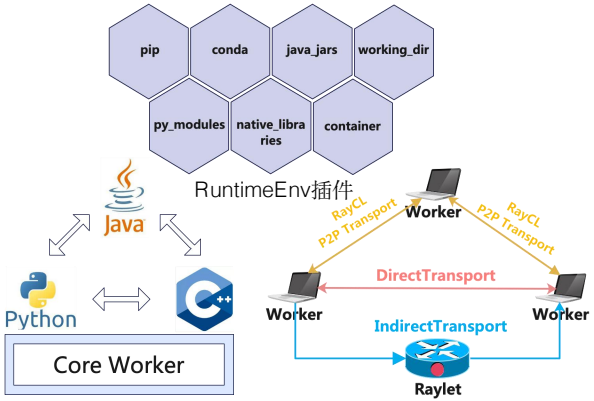


场景	特性	共性
在线服务	• Auto Scaling • 流量调度 • IDC/LDC集群编排	• 面向对象编程API • 跨语言+多语言 • 运行时环境管理 • 服务发现 • 数据存储 • 组件通信 • 异构资源管理 • 任务调度 • 故障恢复 • 运维监控
离线推理	• 异构计算 • 分布式Datasets • AI数据处理算子	
强化学习	• 训推一体 • 多角色协作	
在线学习	• 流处理+训练 • 运行时更新DAG	
图计算	• 静态+动态DAG • 融合计算模式	
媒体计算	• 媒体处理算子 • 高频短任务调度	
函数计算	• 流量调度 • 无状态	

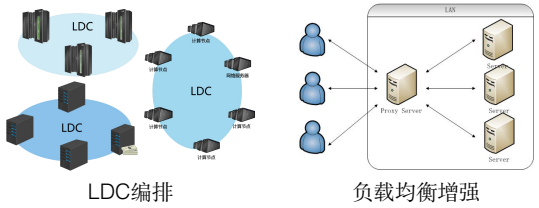
■ 蚂蚁RAY特性增强



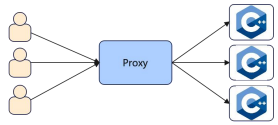
Ray Data



Ray Core



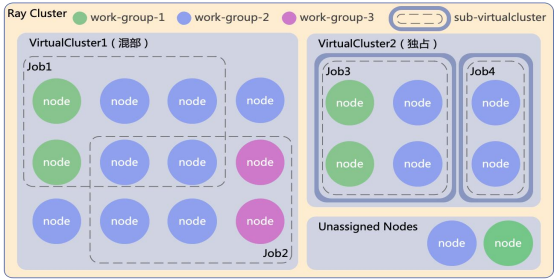
LDC编排　　负载均衡增强



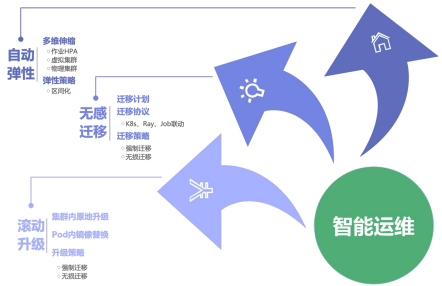
C++服务化

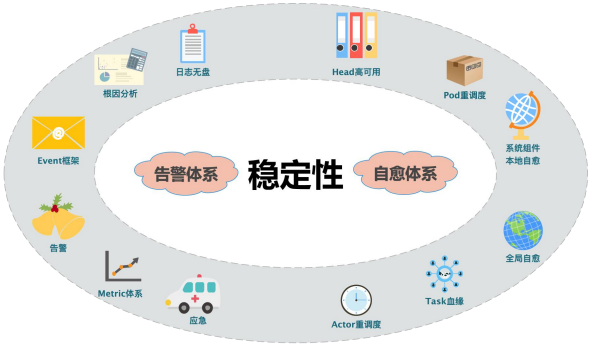
Ray Serve

生产化
规模化

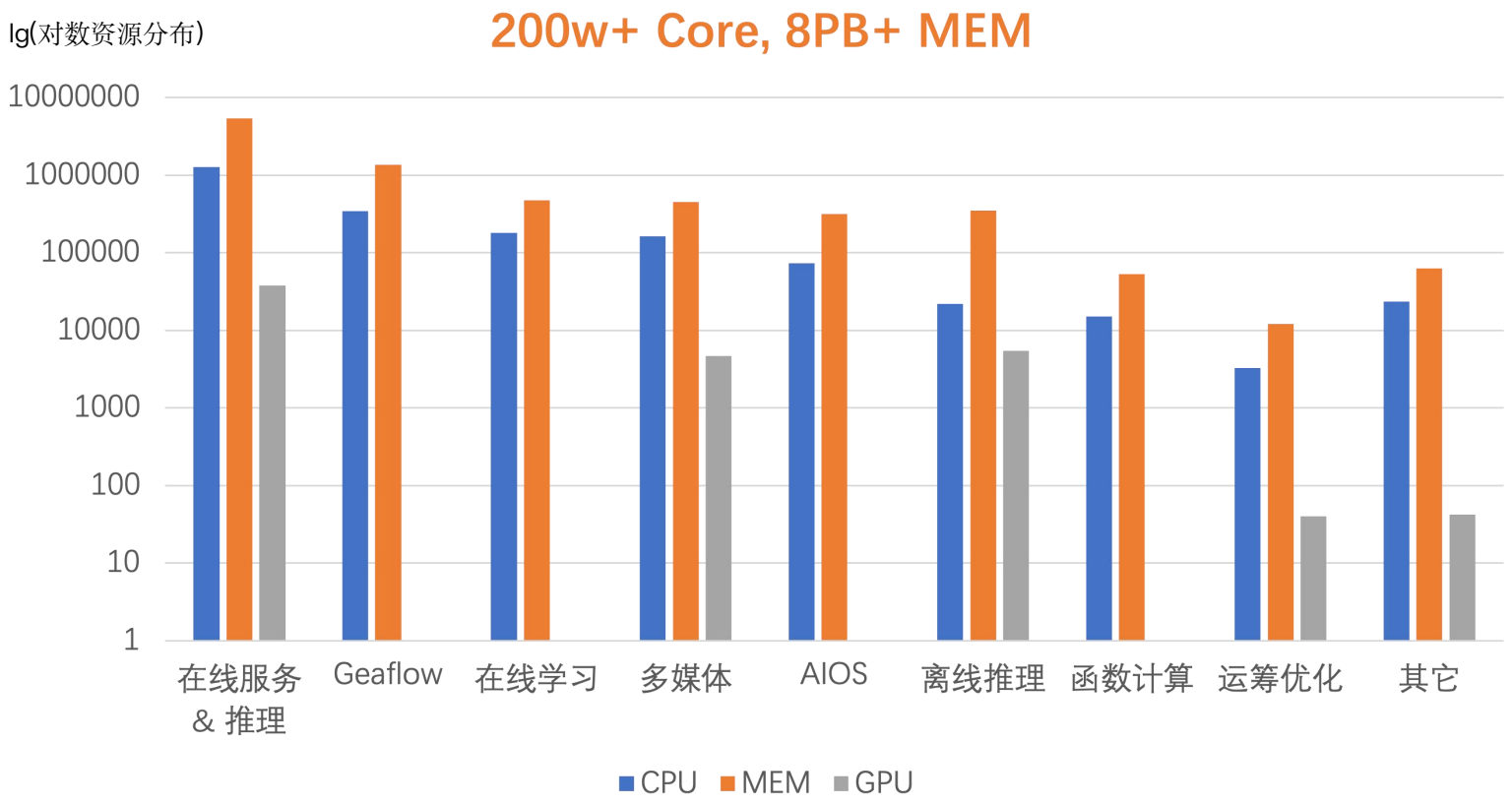


多租户





■ 蚂蚁RAY的规模


lg(对数资源分布)
200w+ Core, 8PB+ MEM
10000000
1000000
100000
10000
1000
100
10
1
在线服务 & 推理
Geaflow
在线学习
多媒体
AIOS
离线推理
函数计算
运筹优化
其它
CPU
MEM
GPU

■ 后训练时代：复杂强化学习系统"黑盒"困境

预训练

系统结构黑盒化

分布式系统组件间交互复杂，算法工程师无法获得全局视图，难以理解多组件间协作关系。

任务分布不透明

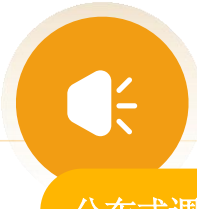
系统各组件分散在不同节点执行不同任务，协作关系复杂，缺乏有效工具帮助理解系统组件分布和资源分配。

性能瓶颈分析难

分布式环境中性能问题来源多样，缺少数据驱动的辅助工具，开发者往往通过猜测和日志分析寻找瓶颈

执行卡点追踪慢

各组件跨节点依赖，执行卡点可能出现在任何节点，需要逐层回溯上下游依赖。



后训练

分布式调试困难

开发者需跨节点收集日志或者代码侵入式添加断点调试语句，每次变更均需重新构建，影响整体的效率。

Flow Insight: 分布式应用可视化分析工具

蚂蚁集团 ANT GROUP | 支付宝

逻辑视图

系统拓扑结构直观展示

分布式调用栈

跨节点调用链路追踪

物理视图

资源分配与使用情况可视化

可视化分布式调试

零代码入侵运行时断点调试

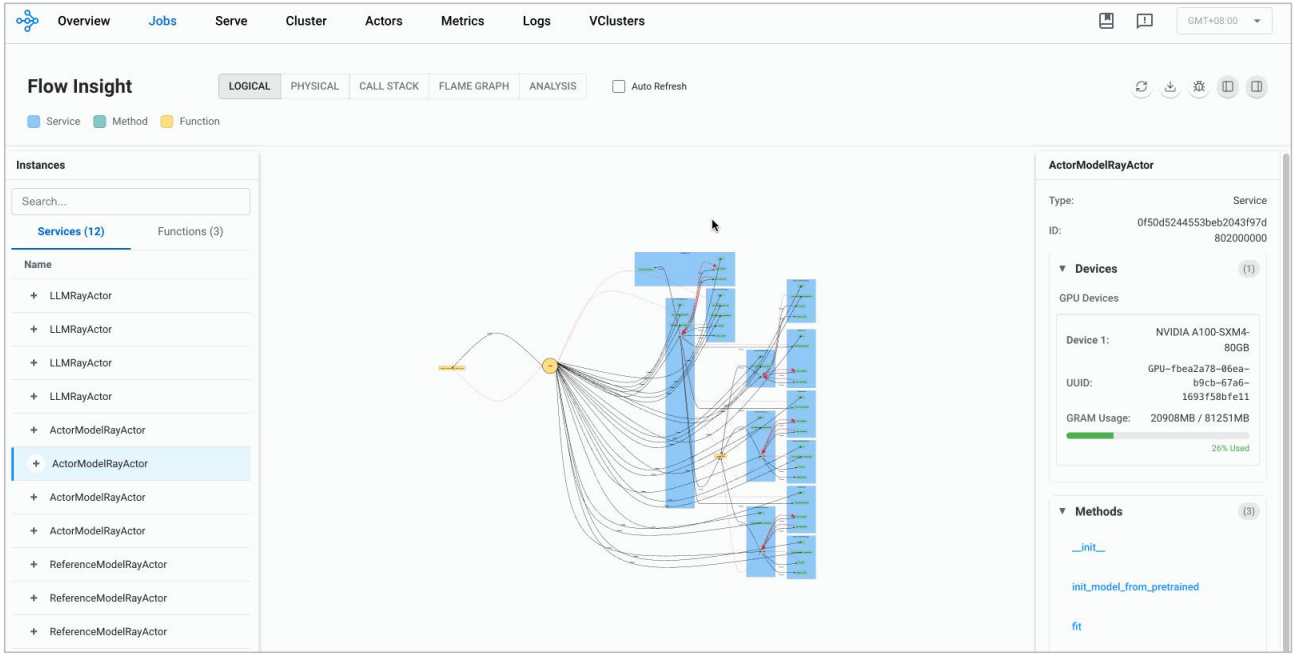
分布式火焰图

性能瓶颈精准定位

AI报告分析

提供基于LLM的系统体检报告

■ 逻辑视图：系统拓扑结构直观展示



全自动渲染系统拓扑

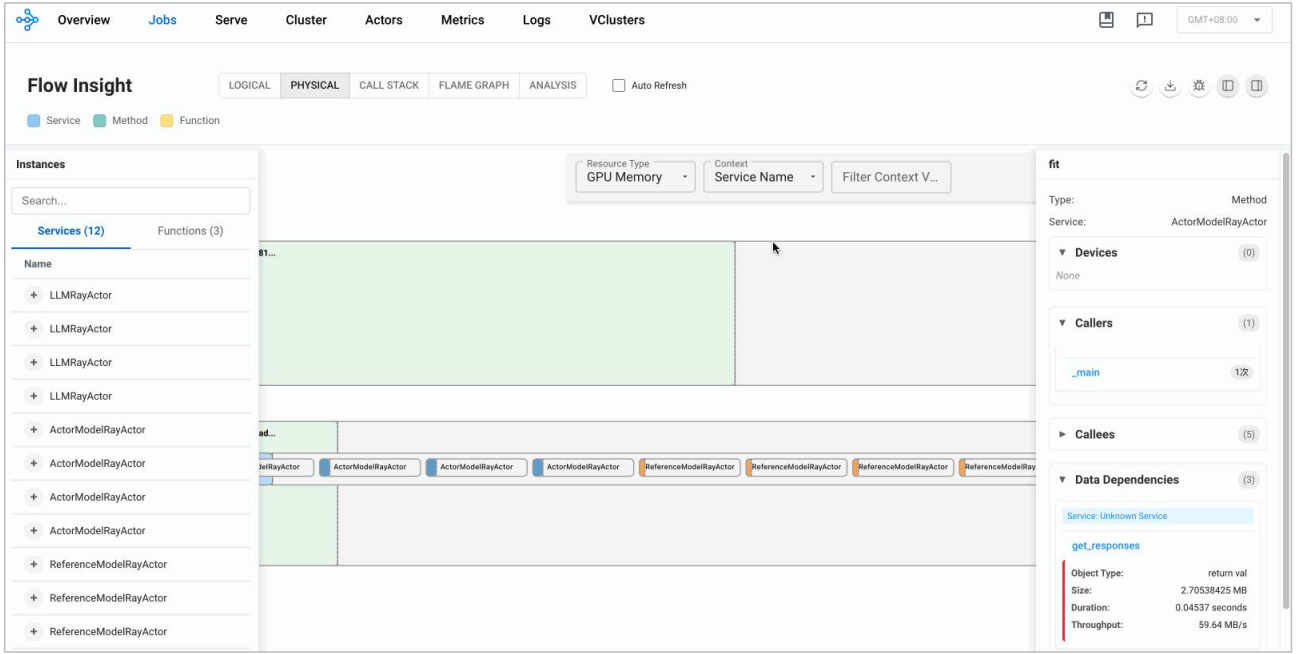
控制流调用关系及频次

数据流吞吐、流向及流速

远程实例和方法分组及检索

上下游调用及数据依赖详情

■ 物理视图：资源分配与使用情况可视化



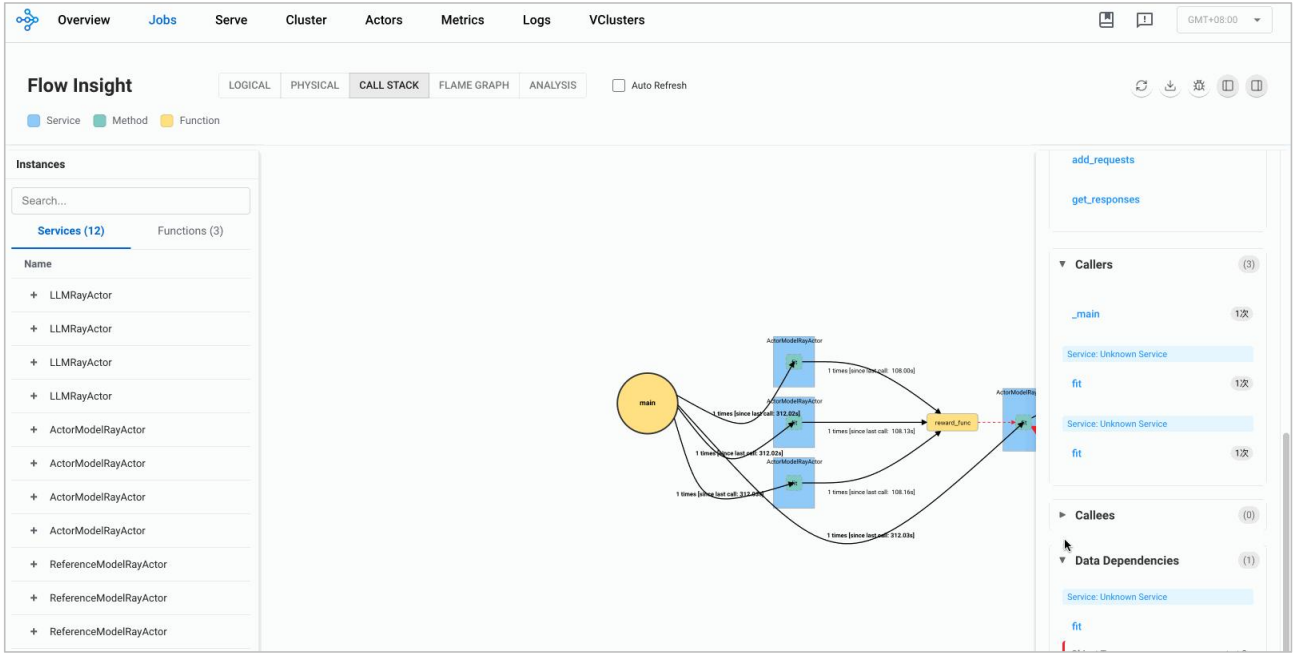
全局节点资源用量监控

多维度资源过滤筛选

标签化实例显示及筛选

EP/PP/DP可视化并行策略

■ 分布式调用栈：跨节点调用链路追踪



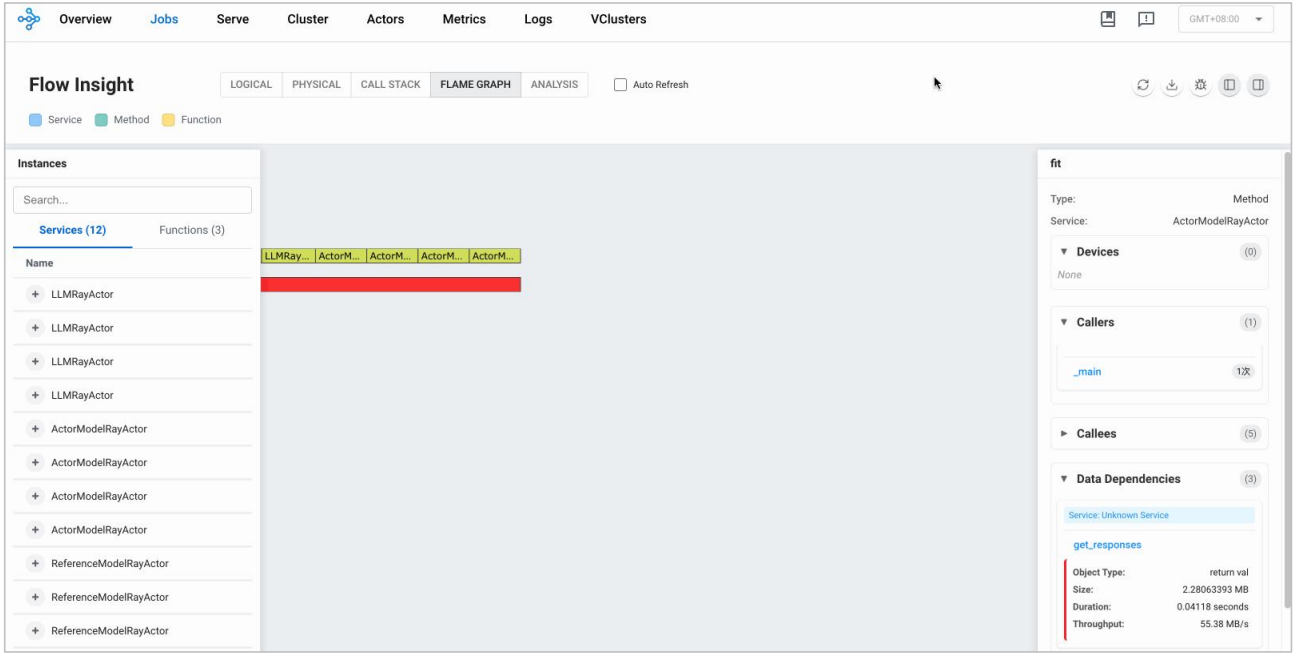
跨节点调用链追踪

全局执行状态

组件调用依赖分析

阻塞点快速定位

■ 分布式火焰图：性能瓶颈精准定位



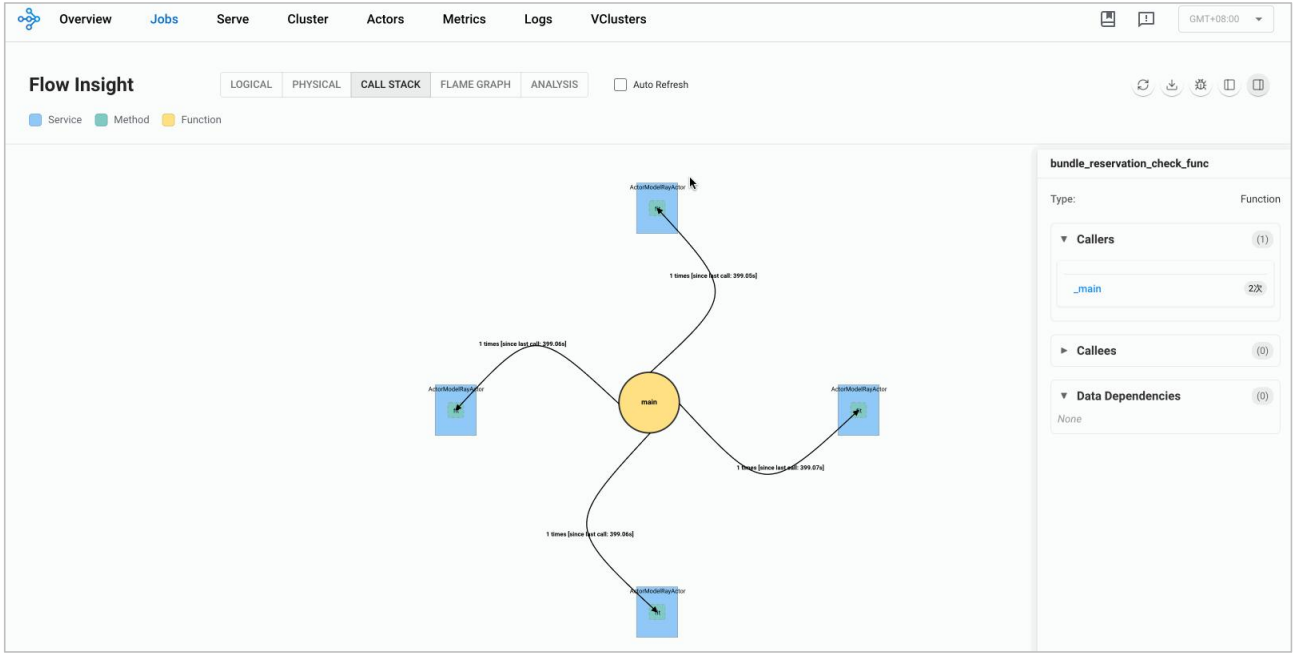
全系统性能剖析

耗时环节定量分析

热点函数定位

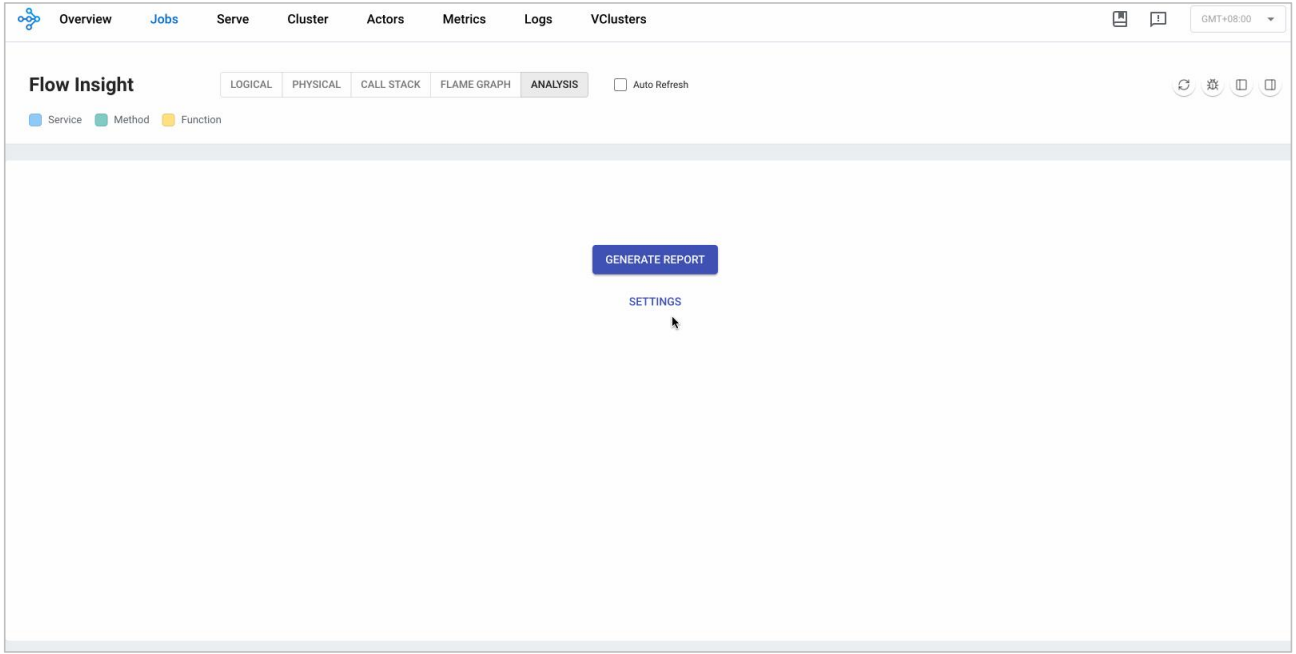
上下文详情透出

■ 分布式调试：零代码入侵运行时断点调试



- 运行时中断
- 源码预览
- 断点管理
- 交互式可视化调试
- Evaluate

■ AI报告分析：提供基于LLM的系统体检报告



- 模型选配
- 性能总结
- 拓扑、调用及数据流分析
- 智能优化建议

谢谢